



# AJAX in JavaScript

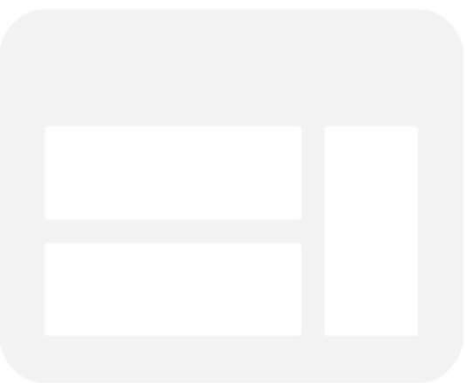
Detailed Notes



 Asynchronous

 JavaScript

 XML / JSON



# Introduction to AJAX

---

## How It Works

### ⇔ What is AJAX?

Asynchronous JavaScript and XML  
Send and receive data from a server **without reloading** the entire web page.

- ✓ Dynamic & quick web page updates

#### ✗ Without AJAX

- Page reloads
  - Server processes request
  - New page loads
- **Slow and inefficient**

#### ✓ With AJAX

- Request in background
  - Server returns data
  - Partial page updates
- **Faster & Better UX ✓**

## Key Technologies Used in AJAX



### Not a Single Technology

AJAX combines multiple web technologies to create dynamic web applications.

 **HTML**

Structure

 **CSS**

Styling

 **JavaScript**

Logic

 **DOM**

Update page content

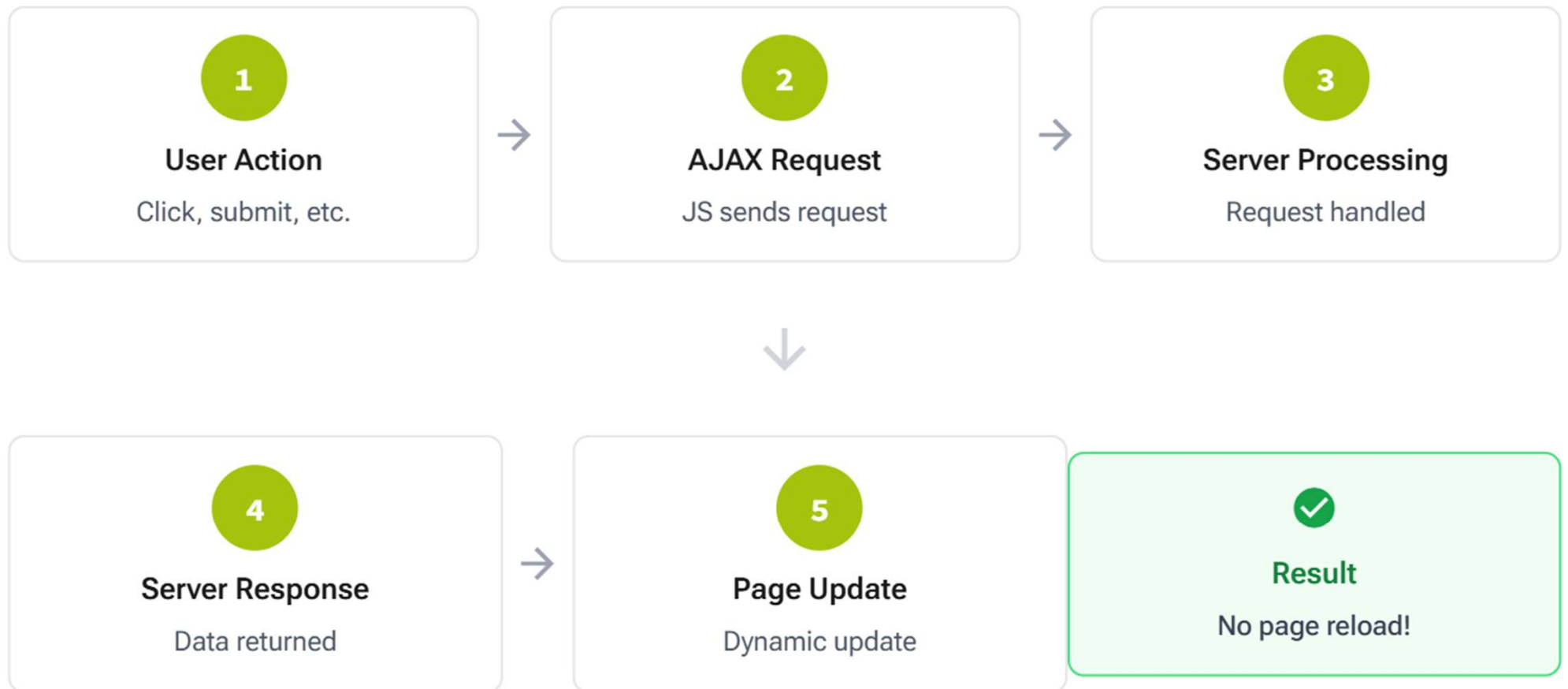
 **XMLHttpRequest**

Communicate with server

 **JSON / XML**

Data format

## How AJAX Works



# ❖ XMLHttpRequest Object

## ⇒ Core Component

AJAX communication happens using the **XMLHttpRequest** object.

- ✓ Send HTTP requests to servers
- ✓ Receive responses asynchronously
- ✓ Update webpage dynamically

## <> Basic Syntax

```
let xhttp = new  
XMLHttpRequest();
```

### i What This Does:

- Creates new XMLHttpRequest instance
- Enables AJAX communication
- Ready to send/receive data

# HTTP AJAX Request Methods

## HTTP Methods



### GET

Retrieve data

Send request to **retrieve data** from server



### POST

Send data

Send request to **send data** to server

## <> GET Request Example

```
let xhr = new XMLHttpRequest();

xhr.open("GET", "data.txt", true);

xhr.onload = function() {
    document.getElementById("demo").innerHTML += xhr.responseText;
};

xhr.send();
```

**open()**

Initialize request

**onload**

Response handler

**send()**

Send request

# HTTP AJAX Request Methods

## HTTP Methods



### GET

Retrieve data

Send request to **retrieve data** from server



### POST

Send data

Send request to **send data** to server

## <> GET Request Example

```
let xhr = new XMLHttpRequest();

xhr.open("GET", "data.txt", true);

xhr.onload = function() {
    document.getElementById("demo").innerHTML += xhr.responseText;
};

xhr.send();
```

**open()**

Initialize request

**onload**

Response handler

**send()**

Send request

## AJAX Example (Complete)

### <> HTML Code

```
<button onclick="loadData()">
  Load Data
</button>

<div id="demo"></div>
```

 Button triggers **loadData()** function

### JS JavaScript Code

```
function loadData() {
  let xhr = new XMLHttpRequest();

  xhr.open("GET", "data.txt", true);

  xhr.onload = function() {
    document.getElementById("demo")
      .innerHTML = this.responseText;
  };

  xhr.send();
}
```

open()

onload

send()



# { } AJAX Using JSON

## ↗ JSON Format

Most AJAX data is exchanged using **JSON** instead of XML.

### <> JSON Data Example

```
{  
  "name": "Vivek",  
  "course": "Web Development"  
}
```

- ✓ Lightweight, easy to parse, native to JavaScript

### JS Fetch API Example

```
fetch("data.json")  
  .then(response => response.json())  
  .then(data => {  
    console.log(data.name);  
  });
```

**fetch()**  
Make request

**json()**  
Parse JSON

**then()**  
Handle data

# AJAX Using Fetch API (Modern Method)

## Modern Alternative

Modern JavaScript uses **Fetch API** instead of XMLHttpRequest.

- ✓ **Promise-based API**
- ✓ **Cleaner syntax** and easier to use
- ✓ **Native support** in modern browsers

## <> Fetch API Example

```
fetch("data.txt")
  .then(response => response.text())
  .then(data => {
    document.getElementById("demo").innerHTML
  });
```

**fetch()**

Make HTTP request

**text()**

Get response text

**then()**

Handle response

## AJAX with Async / Await

### <> Async/Await Syntax

```
async function loadData() {  
  let response = await fetch("data.txt");  
  
  let data = await response.text();  
  
  document.getElementById("demo").innerHTML = c  
}
```

**async**

Async function

**await**

Wait for promise

### Benefits

Modern, readable async code

 **More Readable**  
Synchronous-looking code

 **Error Handling**  
Try/catch support

 **No Callbacks**  
Cleaner code structure

# Advantages of AJAX

## Key Benefits

AJAX revolutionized web development by enabling **dynamic** and **efficient** web applications.



### Enhanced Performance

Faster, smoother, better



### Faster Web Apps

No page reload required



### Better UX

Smooth user interaction



### Reduced Server Load

Only partial updates



### Asynchronous

Background communication

# ⚠ Disadvantages of AJAX

## ⚠ Challenges

While AJAX is powerful, developers must be aware of its **limitations** and **challenges**.



### Important Considerations

Plan and mitigate carefully



### Complex Debugging

Harder to debug



### Security Concerns

Vulnerable to attacks



### JS Dependency

Requires JS enabled



### Browser History

Back button problems

## Real World Examples of AJAX



### Popular Applications

AJAX powers **dynamic experiences** across many of today's most popular websites and applications.



#### Industry Standard

Essential for modern web apps



#### Google Search Suggestions

Real-time search predictions



#### Gmail Loading Emails

Background email fetching



#### Facebook Notifications

Real-time notification updates



#### Instagram Feed Updates

Infinite scroll and new posts



#### Live Chat Applications

Real-time messaging



## ⚡ AJAX vs Traditional Web Requests



### Feature Comparison

Understanding the key differences between **traditional** and **AJAX** approaches.



#### AJAX Advantage

Superior user experience

| ☰ Feature       | 🔄 Traditional | 🔄 AJAX         |
|-----------------|---------------|----------------|
| Page Reload     | ✓ Yes         | ✗ No           |
| Speed           | 🕒 Slower      | ▶▶ Faster      |
| User Experience | 😞 Poor        | 😊 Better       |
| Data Exchange   | 📄 Full Page   | 🔄 Partial Data |

## ⚡ AJAX vs Traditional Web Requests



### Feature Comparison

Understanding the key differences between **traditional** and **AJAX** approaches.



#### AJAX Advantage

Superior user experience

| ☰ Feature       | 🔄 Traditional | 🔄 AJAX         |
|-----------------|---------------|----------------|
| Page Reload     | ✓ Yes         | ✗ No           |
| Speed           | 🕒 Slower      | ▶▶ Faster      |
| User Experience | 😞 Poor        | 😊 Better       |
| Data Exchange   | 📄 Full Page   | 🔄 Partial Data |



# AJAX Workflow Diagram

